CDS 534 Database Management
Group Project: Design a Real-World Database
(5 Topics | ER Diagrams | Correct SQL | Answer Key)

Team Name: _____
Members and Student IDs: _____
Instructor: _____

Deadline: 8:00 pm, 12 November 2026

Submission Checklist

- Each group should have 5–6 students, with one elected team lead (PM).
- Final report (PDF, 10–12 pages, font size 12) due: **8:00 pm, 12 November 2026**.
- Deliverables:
  - PDF report (compulsory);
  - Database build script `.sql` and sample data (screenshot only, compulsory, for verification);
  - ER diagram (vector graphic or in-document drawing) (compulsory).
- File naming: `CDS534_Group_TeamName.pdf`; the title page must list every member's student ID.

1 Assignment Overview

This assignment asks your **test group to design a real-world database system on your own**. Pick **one** of the **5 candidate topics** below (or propose your own with approval) and complete:

1. Describe the business context, stakeholders, and measurable design goals/constraints.
2. Model the entities, attributes, primary keys, foreign keys, and cardinalities with an **ER diagram**.
3. Demonstrate your understanding of **SQL**: give correct table-creation (DDL) and query (DML) statements.
4. Submit the **correct SQL reference** and an **answer key** (see Sections 5 and 6).

2 Five Candidate Topics (Real-World Scenarios)

Each topic gives the business context, the core entities, and a sample ER diagram. The full correct SQL for every topic is in Section 5; the marking notes are in Section 6.

2.1 Topic 1: Campus Second-Hand Textbook Marketplace

Context. Students resell textbooks at semester end. The platform matches buyers and sellers, recording users, listings, categories, orders, and order items. **Core entities:** User, Category, Book, Order, OrderItem. **Key relationships:** a category contains many books (1:N); a user lists many books and places many orders (1:N); an order has many items, each item references one book (1:N).

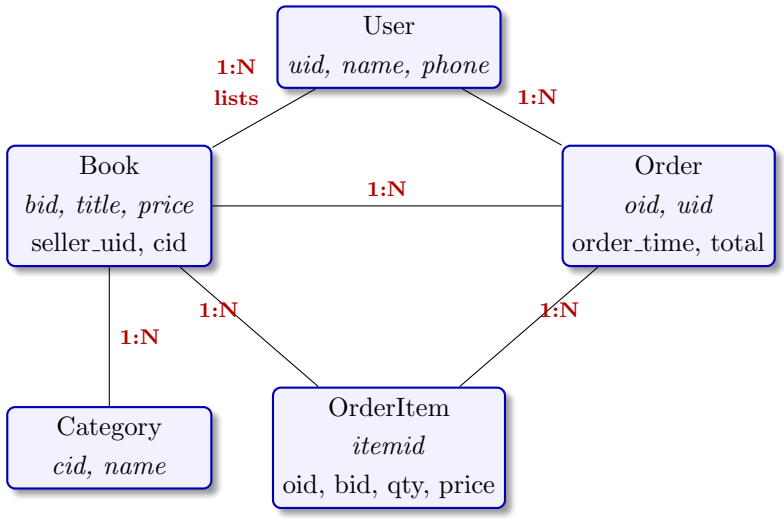


Figure 1: Topic 1 sample ER diagram (all 1:N; N:M resolved by OrderItem)

2.2 Topic 2: Community Fresh-Food Next-Day Delivery

Context. A community group-buy / grocery platform managing stores, products, customers, riders, and orders. **Core entities:** Store, Product, Customer, Rider, Order, OrderItem. **Key relationships:** a store stocks many products (1:N); a customer places many orders (1:N); an order is delivered by one rider (N:1); an order contains many products and a product appears in many orders $\Rightarrow$ `Order`↔`Product` is `N:M`, resolved by `OrderItem`.

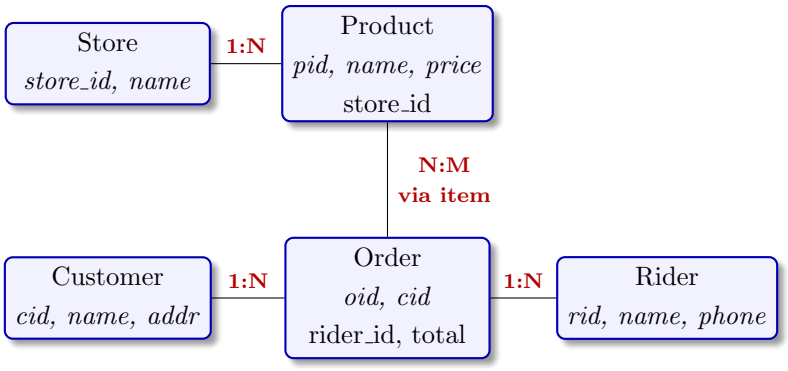


Figure 2: Topic 2 sample ER diagram (N:M Order-Product resolved by OrderItem)

2.3 Topic 3: Gym Membership & Class Management

Context. A gym tracks members, membership plans, check-ins, personal trainers, and group-class enrollment. **Core entities:** Member, Membership, CheckIn, Trainer, Class, Enrollment (associative). **Key relationships:** a member buys one membership (N:1); a member checks in many times (1:N); a trainer teaches many classes (1:N); a member enrolls in many classes and a class holds many members ⇒ `Member`↔`Class` is `N:M`, resolved by associative entity `Enrollment`.

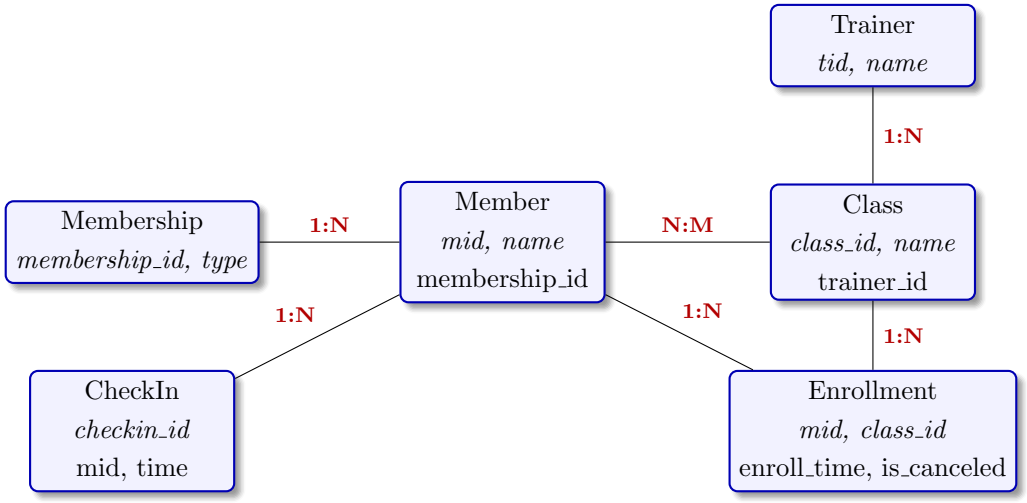


Figure 3: Topic 3 sample ER diagram (N:M Member-Class resolved by Enrollment)

2.4 Topic 4: Library Lending & Overdue Fines

Context. The system manages readers, the book catalogue, lending records, categories, and librarians; overdue fines are computed from the due date. **Core entities:** Reader, Book, Borrow, Category, Librarian. **Key relationships:** a book belongs to one category (N:1); a reader borrows many books (1:N); a book is borrowed many times (1:N); a librarian processes many loans (1:N). *All relationships are 1:N.*

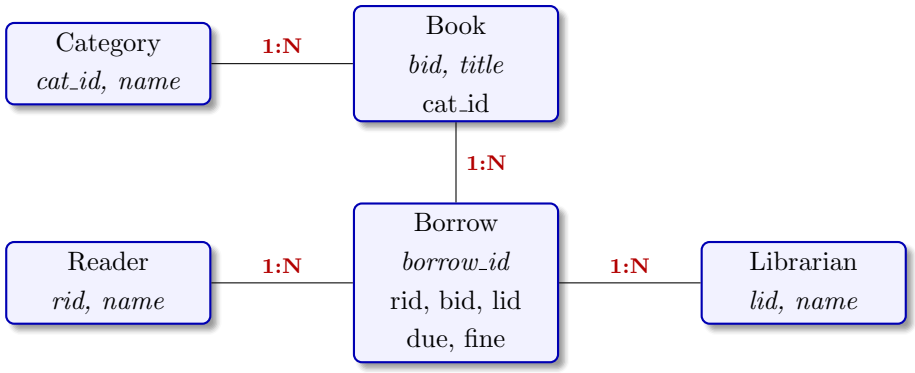


Figure 4: Topic 4 sample ER diagram (all 1:N)

2.5 Topic 5: Ride-Hailing Order & Trip Dispatch

Context. Passengers request trips, drivers accept them, fares are charged and paid; each driver is bound to one vehicle. **Core entities:** Passenger, Driver, Vehicle, Trip, Payment. **Key relationships:** a passenger requests many trips (1:N); a driver serves many trips (1:N); a driver is bound to one vehicle (N:1); a trip corresponds to exactly one payment (1:1).

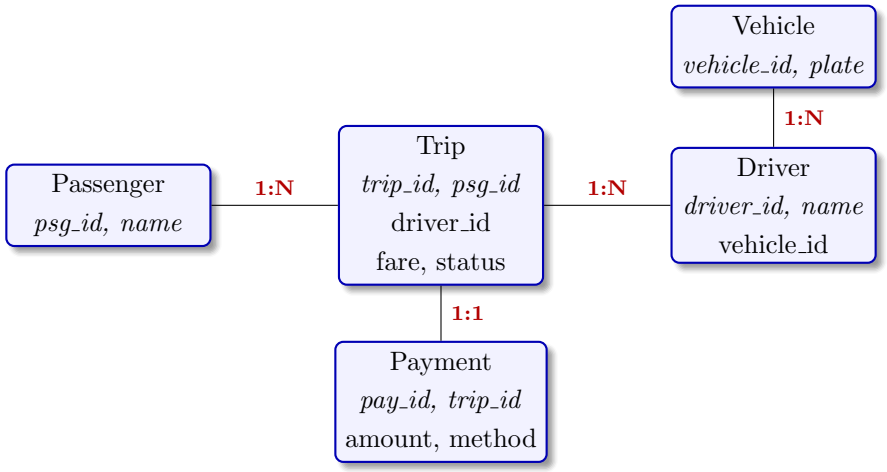


Figure 5: Topic 5 sample ER diagram (1:1 Trip-Payment; N:1 Driver-Vehicle)

3 SQL Understanding Questions

Answer the following in your report to demonstrate your command of SQL:

1. Explain the role of a `PRIMARY KEY` versus a `FOREIGN KEY`, and how they together enforce referential integrity.
2. Summarize the three normal forms (1NF / 2NF / 3NF) and give an example showing why transitive dependencies should be removed.

3. What is the difference between `INNER JOIN`, `LEFT JOIN`, and `RIGHT JOIN`? Give one scenario where `LEFT JOIN` is mandatory.
4. For "find users who bought a product of category X", how would you write it with a subquery versus a `JOIN`? Which is usually more efficient, and why?
5. Explain the ACID properties of transactions; give a "two concurrent transfers" example showing why isolation and atomicity are needed.
6. How does an index speed up queries, and why should you not index every column of every table?
7. What do `GROUP BY` and `HAVING` do, and how do they differ from `WHERE` in terms of when filtering happens?
8. What is the difference between a `VIEW` and a `STORED PROCEDURE`, and when is each appropriate?

4 ER Diagram Conventions

- Rectangle = entity; attributes are written inside the box; the **primary key is in italics**.
- Label each link with its cardinality: `1:N`, `N:M`, `1:1`.
- A many-to-many relationship **must** be resolved through an *associative entity* (e.g. `OrderItem`, `Enrollment`) split into two 1:N links.
- Foreign-key attributes are marked in the child entity as `*_id`.

The ER diagrams in Section 2 already follow these rules and may be reused or redrawn.

5 Correct SQL Reference Implementations

Complete, runnable DDL and three illustrative queries are given for every topic.

Topic 1 – Campus Second-Hand Textbook Marketplace (DDL)

```
CREATE TABLE User (
  uid    INT PRIMARY KEY,
  name   VARCHAR(40) NOT NULL,
  phone  VARCHAR(20) UNIQUE
);
CREATE TABLE Category (
  cid    INT PRIMARY KEY,
  name   VARCHAR(50) NOT NULL
);
CREATE TABLE Book (
  bid         INT PRIMARY KEY,
```

```
  title       VARCHAR(120) NOT NULL,
  price       DECIMAL(8,2) CHECK (price >= 0),
  seller_uid  INT NOT NULL,
  cid         INT NOT NULL,
  FOREIGN KEY (seller_uid) REFERENCES User(uid),
  FOREIGN KEY (cid)        REFERENCES Category(cid)
);
CREATE TABLE `Order` (
  oid         INT PRIMARY KEY,
  uid         INT NOT NULL,
  order_time  DATETIME DEFAULT CURRENT_TIMESTAMP,
  total       DECIMAL(10,2) CHECK (total >= 0),
  FOREIGN KEY (uid) REFERENCES User(uid)
);
CREATE TABLE OrderItem (
  itemid INT PRIMARY KEY,
  oid    INT NOT NULL,
  bid    INT NOT NULL,
  qty    INT CHECK (qty > 0),
  price  DECIMAL(8,2) CHECK (price >= 0),
  FOREIGN KEY (oid) REFERENCES `Order`(oid),
  FOREIGN KEY (bid) REFERENCES Book(bid)
);
```

Listing 1: Topic 1 DDL

Queries.

1. Books on sale per category (empty categories kept via `LEFT JOIN`):

```
SELECT c.name AS category, COUNT(b.bid) AS books
FROM Category c LEFT JOIN Book b ON c.cid = b.cid
GROUP BY c.cid, c.name;
```

Listing 2: Topic 1 Q1

2. Each user's order count and total spent:

```
SELECT u.name, COUNT(o.oid) AS orders, SUM(o.total) AS spent
FROM User u JOIN `Order` o ON u.uid = o.uid
GROUP BY u.uid, u.name;
```

Listing 3: Topic 1 Q2

3. Top 5 best-selling books:

```
SELECT b.title, SUM(oi.qty) AS sold
FROM Book b JOIN OrderItem oi ON b.bid = oi.bid
GROUP BY b.bid, b.title
```

```
ORDER BY sold DESC
LIMIT 5;
```

Listing 4: Topic 1 Q3

Topic 2 – Community Fresh-Food Next-Day Delivery (DDL)

```
CREATE TABLE Store (
  store_id INT PRIMARY KEY,
  name     VARCHAR(60) NOT NULL
);
CREATE TABLE Product (
  pid      INT PRIMARY KEY,
  name     VARCHAR(80) NOT NULL,
  price    DECIMAL(8,2) CHECK (price >= 0),
  store_id INT NOT NULL,
  FOREIGN KEY (store_id) REFERENCES Store(store_id)
);
CREATE TABLE Customer (
  cid  INT PRIMARY KEY,
  name VARCHAR(40) NOT NULL,
  addr VARCHAR(120)
);
CREATE TABLE Rider (
  rid   INT PRIMARY KEY,
  name  VARCHAR(40) NOT NULL,
  phone VARCHAR(20)
);
CREATE TABLE `Order` (
  oid        INT PRIMARY KEY,
  cid        INT NOT NULL,
  rider_id   INT NOT NULL,
  total      DECIMAL(10,2) CHECK (total >= 0),
  order_time DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (cid)      REFERENCES Customer(cid),
  FOREIGN KEY (rider_id) REFERENCES Rider(rid)
);
CREATE TABLE OrderItem (
  itemid INT PRIMARY KEY,
  oid    INT NOT NULL,
  pid    INT NOT NULL,
  qty    INT CHECK (qty > 0),
  price  DECIMAL(8,2) CHECK (price >= 0),
  FOREIGN KEY (oid) REFERENCES `Order`(oid),
```

```
  FOREIGN KEY (pid) REFERENCES Product(pid)
);
```

Listing 5: Topic 2 DDL

Queries.

1. Total revenue per store (join through OrderItem):

```
SELECT s.name AS store, SUM(oi.qty * oi.price) AS revenue
FROM Store s
JOIN Product p  ON p.store_id = s.store_id
JOIN OrderItem oi ON oi.pid = p.pid
GROUP BY s.store_id, s.name;
```

Listing 6: Topic 2 Q1

2. Orders delivered by each rider:

```
SELECT r.name, COUNT(o.oid) AS deliveries
FROM Rider r LEFT JOIN `Order` o ON o.rider_id = r.rid
GROUP BY r.rid, r.name;
```

Listing 7: Topic 2 Q2

3. Customers who bought a specific product (e.g. `pid = 7`):

```
SELECT DISTINCT c.name
FROM Customer c
JOIN `Order` o ON o.cid = c.cid
JOIN OrderItem oi ON oi.oid = o.oid
WHERE oi.pid = 7;
```

Listing 8: Topic 2 Q3

Topic 3 – Gym Membership & Class Management (DDL)

```
CREATE TABLE Membership (
  membership_id INT PRIMARY KEY,
  type          VARCHAR(40) NOT NULL
);
CREATE TABLE Member (
  mid           INT PRIMARY KEY,
  name          VARCHAR(40) NOT NULL,
  membership_id INT NOT NULL,
  FOREIGN KEY (membership_id) REFERENCES Membership(membership_id)
);
CREATE TABLE Trainer (
```

```
  tid  INT PRIMARY KEY,
  name VARCHAR(40) NOT NULL
);
CREATE TABLE Class (
  class_id   INT PRIMARY KEY,
  name       VARCHAR(60) NOT NULL,
  trainer_id INT NOT NULL,
  FOREIGN KEY (trainer_id) REFERENCES Trainer(tid)
);
CREATE TABLE CheckIn (
  checkin_id INT PRIMARY KEY,
  mid        INT NOT NULL,
  time       DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (mid) REFERENCES Member(mid)
);
CREATE TABLE Enrollment (
  mid          INT NOT NULL,
  class_id     INT NOT NULL,
  enroll_time  DATETIME DEFAULT CURRENT_TIMESTAMP,
  is_canceled  TINYINT(1) DEFAULT 0,
  PRIMARY KEY (mid, class_id),
  FOREIGN KEY (mid)      REFERENCES Member(mid),
  FOREIGN KEY (class_id) REFERENCES Class(class_id)
);
```

Listing 9: Topic 3 DDL

Queries.

1. Members enrolled in more than 3 classes (composite PK + `HAVING`):

```
SELECT m.name, COUNT(e.class_id) AS classes
FROM Member m JOIN Enrollment e ON e.mid = m.mid
WHERE e.is_canceled = 0
GROUP BY m.mid, m.name
HAVING COUNT(e.class_id) > 3;
```

Listing 10: Topic 3 Q1

2. Each trainer's class count:

```
SELECT t.name, COUNT(c.class_id) AS classes
FROM Trainer t LEFT JOIN Class c ON c.trainer_id = t.tid
GROUP BY t.tid, t.name;
```

Listing 11: Topic 3 Q2

3. Total check-ins per membership type:

```sql
SELECT ms.type, COUNT(ci.checkin_id) AS checkins
FROM Membership ms
JOIN Member m ON m.membership_id = ms.membership_id
JOIN CheckIn ci ON ci.mid = m.mid
GROUP BY ms.membership_id, ms.type;
```

Listing 12: Topic 3 Q3

Topic 4 – Library Lending & Overdue Fines (DDL)

```sql
CREATE TABLE Category (
  cat_id INT PRIMARY KEY,
  name   VARCHAR(50) NOT NULL
);
CREATE TABLE Book (
  bid   INT PRIMARY KEY,
  title VARCHAR(120) NOT NULL,
  cat_id INT NOT NULL,
  FOREIGN KEY (cat_id) REFERENCES Category(cat_id)
);
CREATE TABLE Reader (
  rid  INT PRIMARY KEY,
  name VARCHAR(40) NOT NULL
);
CREATE TABLE Librarian (
  lid  INT PRIMARY KEY,
  name VARCHAR(40) NOT NULL
);
CREATE TABLE Borrow (
  borrow_id   INT PRIMARY KEY,
  rid         INT NOT NULL,
  bid         INT NOT NULL,
  lid         INT NOT NULL,
  borrow_date DATE,
  due         DATE NOT NULL,
  return_date DATE,
  fine        DECIMAL(8,2) DEFAULT 0,
  FOREIGN KEY (rid) REFERENCES Reader(rid),
  FOREIGN KEY (bid) REFERENCES Book(bid),
  FOREIGN KEY (lid) REFERENCES Librarian(lid)
);
```

Listing 13: Topic 4 DDL

Queries.

1. Currently overdue books (return_date is NULL and due passed):

```sql
SELECT b.title, r.name, br.due
FROM Borrow br
JOIN Book b ON b.bid = br.bid
JOIN Reader r ON r.rid = br.rid
WHERE br.return_date IS NULL AND br.due < CURDATE();
```

Listing 14: Topic 4 Q1

2. Update fine for overdue records (rate = \$1/day) using `GREATEST`:

```sql
UPDATE Borrow
SET fine = GREATEST(0, DATEDIFF(CURDATE(), due)) * 1.00
WHERE return_date IS NULL AND due < CURDATE();
```

Listing 15: Topic 4 Q2

3. Number of loans handled by each librarian:

```sql
SELECT l.name, COUNT(br.borrow_id) AS loans
FROM Librarian l LEFT JOIN Borrow br ON br.lid = l.lid
GROUP BY l.lid, l.name;
```

Listing 16: Topic 4 Q3

Topic 5 – Ride-Hailing Order & Trip Dispatch (DDL)

```sql
CREATE TABLE Passenger (
  psg_id INT PRIMARY KEY,
  name   VARCHAR(40) NOT NULL
);
CREATE TABLE Vehicle (
  vehicle_id INT PRIMARY KEY,
  plate      VARCHAR(20) NOT NULL UNIQUE
);
CREATE TABLE Driver (
  driver_id  INT PRIMARY KEY,
  name       VARCHAR(40) NOT NULL,
  vehicle_id INT NOT NULL UNIQUE,
  FOREIGN KEY (vehicle_id) REFERENCES Vehicle(vehicle_id)
);
CREATE TABLE Trip (
  trip_id    INT PRIMARY KEY,
  psg_id     INT NOT NULL,
  driver_id  INT NOT NULL,
  fare       DECIMAL(8,2),
```

```
  status      VARCHAR(20),
  start_time DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (psg_id)    REFERENCES Passenger(psg_id),
  FOREIGN KEY (driver_id) REFERENCES Driver(driver_id)
);
CREATE TABLE Payment (
  pay_id   INT PRIMARY KEY,
  trip_id  INT NOT NULL UNIQUE,
  amount   DECIMAL(8,2) CHECK (amount >= 0),
  method   VARCHAR(20),
  FOREIGN KEY (trip_id) REFERENCES Trip(trip_id)
);
```

Listing 17: Topic 5 DDL

Queries.

1. Completed trips with their payment (1:1 join):

```
SELECT t.trip_id, p.name AS passenger, t.fare, py.amount, py.method
FROM Trip t
JOIN Passenger p ON p.psg_id = t.psg_id
JOIN Payment py ON py.trip_id = t.trip_id
WHERE t.status = 'completed';
```

Listing 18: Topic 5 Q1

2. Drivers not yet paid for a completed trip (anti-join):

```
SELECT d.name
FROM Driver d
JOIN Trip t ON t.driver_id = d.driver_id
WHERE t.status = 'completed'
  AND NOT EXISTS (SELECT 1 FROM Payment py WHERE py.trip_id = t.
     trip_id);
```

Listing 19: Topic 5 Q2

3. Total fare earned per vehicle:

```
SELECT v.plate, SUM(t.fare) AS earned
FROM Vehicle v
JOIN Driver d ON d.vehicle_id = v.vehicle_id
JOIN Trip t   ON t.driver_id = d.driver_id
GROUP BY v.vehicle_id, v.plate;
```

Listing 20: Topic 5 Q3

6 Answer Key

SQL Conceptual Questions – Answers

1. **Primary key (PK) vs Foreign key (FK).** A *primary key* uniquely identifies each row in a table and must be `NOT NULL` and `UNIQUE`; it is the address used to locate a record. A *foreign key* is a column (or set of columns) in a child table whose values must exist as a primary key in a referenced parent table. The FK constraint enforces *referential integrity*: it forbids orphan rows (a child row pointing to a non-existent parent) and, with `ON DELETE/UPDATE CASCADE|RESTRICT|SET NULL`, controls what happens when the parent row changes. Example: `Book.seller_uid` references `User.uid`; you cannot insert a book whose seller does not exist.

2. **Three normal forms (1NF / 2NF / 3NF).**
   - *1NF:* every column holds atomic (indivisible) values and there are no repeating groups / multi-valued attributes. A cell must not contain a list like `"Math, Physics"`.
   - *2NF:* the table is in 1NF and every non-key attribute depends on the *whole* candidate key, not just part of it. This matters only when the PK is composite. A partial dependency (e.g. `(order_id, product_id)` where `product_name` depends only on `product_id`) must be removed by splitting the table.
   - *3NF:* the table is in 2NF and there is *no transitive dependency* – a non-key attribute must not depend on another non-key attribute.

   Why remove transitive dependencies? Consider `student_id -> dept -> dean`. If the dean is stored on the student row, changing a department's dean forces updating many student rows (update anomaly), and deleting the only student in a department loses the dean information (deletion anomaly). Decompose into `Student(student_id, dept)` and `Department(dept, dean)`.

3. `INNER JOIN` **vs** `LEFT JOIN` **vs** `RIGHT JOIN.** `INNER JOIN` returns only rows with a match in *both* tables. `LEFT JOIN` returns every row of the left table, filling the right-table columns with `NULL` when there is no match. `RIGHT JOIN` is the mirror image. **Must-use-LEFT-JOIN scenario:** "list every customer together with their order count, *including customers who have placed zero orders*." If you `INNER JOIN Customer` to `Order`, zero-order customers disappear; `Customer LEFT JOIN Order` keeps them and `COUNT(o.oid)` yields 0.

4. **Subquery vs** `JOIN` **for "users who bought a product of category X".**
   - Subquery (uncorrelated):

```
SELECT u.name
FROM User u
WHERE u.uid IN (
  SELECT o.uid FROM `Order` o
  JOIN OrderItem oi ON o.oid = oi.oid
```

```
  JOIN Book b ON oi.bid = b.bid
  WHERE b.cid = <X>
);
```

Listing 21: Subquery version

- `JOIN` version:

```
SELECT DISTINCT u.name
FROM User u
JOIN `Order` o ON u.uid = o.uid
JOIN OrderItem oi ON o.oid = oi.oid
JOIN Book b ON oi.bid = b.bid
WHERE b.cid = <X>;
```

Listing 22: JOIN version

The `JOIN` typically lets the optimizer use indexes on the join columns and avoids re-evaluating a nested query per row; `DISTINCT` removes duplicates. For very large sets, `EXISTS` (a semi-join) is often even better than `IN` because it stops at the first match.

5. **ACID properties (transactions).**
   - *Atomicity:* the transaction is all-or-nothing; a failure rolls back every change.
   - *Consistency:* a committed transaction leaves the database in a valid state that satisfies all declared constraints.
   - *Isolation:* concurrent transactions do not see each other's intermediate state. Levels: READ UNCOMMITTED / READ COMMITTED / REPEATABLE READ / SERIALIZABLE trade throughput for stronger guarantees (dirty read, non-repeatable read, phantom read).
   - *Durability:* once committed, the change survives a crash (redo log).

   Two concurrent transfers: without *atomicity*, debiting A might succeed while crediting B fails, losing money. Without *isolation*, both transactions read the same starting balance (e.g. 100), each adds 50, and the final balance is 150 instead of 200 – the classic *lost update*.
6. **Indexes – why they help and why not everywhere.** An index is an auxiliary ordered structure (usually a B+ tree) that maps key values to row locations, turning a full table scan ($O(n)$) into a tree walk ($O(\log n)$) for `WHERE`, `JOIN` keys, and `ORDER BY`. We cannot index *every* column of *every* table because: (a) each index consumes storage; (b) every `INSERT/UPDATE/DELETE` must also maintain the index, slowing writes; (c) too many indexes confuse the optimizer and waste buffer-pool memory. Index high-selectivity columns (e.g. `uid`), and use composite indexes with the leftmost-prefix rule.
7. **`GROUP BY` / `HAVING` vs `WHERE`.** `WHERE` filters individual *rows before* grouping. `GROUP BY` buckets the surviving rows and computes aggregates (`COUNT/SUM/AVG`). `HAVING` filters the re-

sulting *groups after* aggregation, so it may reference aggregates while `WHERE` cannot. Example – categories with more than 10 books:

```
SELECT c.name, COUNT(b.bid) AS n
FROM Category c LEFT JOIN Book b ON c.cid = b.cid
GROUP BY c.cid, c.name
HAVING COUNT(b.bid) > 10;
```

Listing 23: HAVING filters groups

8. **View vs Stored procedure.** A *view* is a saved `SELECT` query exposed as a virtual table; it is used for access control (expose only certain columns/rows), simplifying complex queries, and logical independence. Under limits it is updatable. A *stored procedure* is a precompiled server-side program with parameters, control flow, and DML that encapsulates business logic or batch jobs and is invoked with `CALL`. A view is queried like a table; a procedure *does* something. (A scalar/table-valued *function* is the in-between option that returns a value and can be used inside expressions.)

Per-Topic Answer Notes

- **Topic 1:** entities User / Category / Book / Order / OrderItem; all relationships are 1:N; `OrderItem` is the associative entity resolving Book↔Order. The DDL is in Section 5. Score: correct PK/FK, `OrderItem` as the resolving entity, `LEFT JOIN` in Q1, and `GROUP BY` on the non-aggregated select columns.
- **Topic 2:** the make-or-break item is introducing `OrderItem` to break the `N:M` Order↔Product; without it the schema is wrong. Rider is the 1:N side of Order.
- **Topic 3:** Member↔Class is `N:M` and *must* use the `Enrollment` associative entity with a *composite primary key* `(mid, class_id)`. Storing `enroll_time` / `is_canceled` as relationship attributes is a plus.
- **Topic 4:** `fine` lives on `Borrow` and is computed by the application or a trigger as `GREATEST(0, DATEDIFF(return_date, due)) * rate`; `return_date IS NULL` correctly detects an open loan.
- **Topic 5:** `Trip`↔`Payment` is 1:1 (enforced by `trip_id UNIQUE` on Payment); `Driver`↔`Vehicle` is N:1 (enforced by `Driver.vehicle_id UNIQUE`). The anti-join in Q2 should use `NOT EXISTS` (or `LEFT JOIN ... WHERE pay_id IS NULL`).

Grading Rubric (25 points)

- Problem framing & significance (4 pts): realistic scenario (2); measurable goals/constraints (2).

- Methodology (9 pts): clear problem statement (3); explicit, justified consistency/availability/latency trade-offs (3); architecture supports the workload (3).
- Evaluation & evidence (9 pts): sensible workload/dataset/baseline (3); correct, clear result tables (3); justified conclusions (3).
- Writing & ER/SQL correctness (3 pts): readable figures, correct SQL, ER follows the 1:N / N:M / 1:1 conventions above (3).